Athens University of Economics and Business

Department of Informatics

COURSE: **PARALLEL PROGRAMMING**

Spring Semester 2025-26

ASSIGNMENT TITLE

Experimenting Different Methods for Workload Distribution with OpenMP in C++

Name:	**MARAKIS MICHAIL**
Semester:	**6th**
Email:	**p3230267@aueb.gr**
Prof:	**Dr. VASILAKIS ANDREAS-ALEXANDROS**

TABLE OF CONTENTS

Περιεχόμενα

1 Introduction

The second assignment in this Parallel Programming course focuses on the use of OpenMP API in C++ for the parallelization of numerical integration. More specifically, it involves the implementation and evaluation of different workload distribution methods in an integral computation application. These approaches include **parallelization using loops** (with and without reduction), **different scheduling policies** (static, dynamic, and guided), and **task-based parallelism** using OpenMP tasks.

The goal is to analyze their performance and determine the most efficient approach under different circumstances. For example, experiment with different #threads used, different chunk size and different num_tasks. The integral method on which these methods are tested is the Trapezoidal Rule for numerical integration:

$$\int_a^b f(x)dx \approx \frac{h}{2}[f(a) + f(b) + 2\sum_{i=1}^{n-1} f(a + ih)],\ h = (b - a)/n$$

1.1 Setup

Metrics were conducted on the following system:

Computer	Processor	Cores	Compiler	O.S.
DESKTOP-MU9QIDD	**Intel i5 vPro 8th Gen**	**4**	**gcc v15.2.0**	**Windows 11**

1.2 Compile and run

First find the name of the file(file_name) where the code is and run this command to compile:

g++ -std=c++17 -fopenmp <file_name>.cpp -o <file_name>

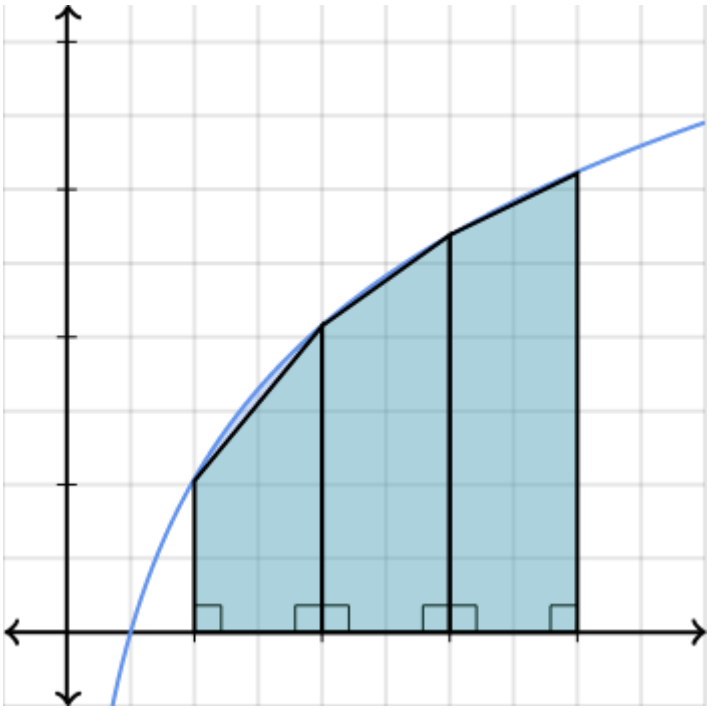
After a successful compilation an .exe file has been created. To run it use the command:

.\<file_name>.exe

1.3 The Problem

The problem is based on approximating an integral using the Riemann sum approach, specifically through the Trapezoidal Rule. According to the definition of the Riemann integral, the continuous space [a,b] is divided into many small subspaces of width $h = \frac{b-a}{N}$, and the integral is approximated as the sum of the function values over these subspaces. As the number of subspaces increases($N \to \infty$) the approximation becomes more accurate and converges to the exact

value of the integral. In the Trapezoidal Rule, the area is approximated using trapezoids instead of rectangles, providing better accuracy. This formulation makes the problem very suitable for parallelization, since each part of the sum is independent and can be computed by different threads, theoretically allowing faster execution compared to the sequential approach.



(Application of the Trapezoid Riemann Integral)

2 Parallelism with Loop

The main idea when we try to implement Parallelism with Loop is that we take a hard to compute loop and distribute its iterations among multiple threads, so that they execute the loop concurrently and not sequentially. This is achieved using the **#pragma omp for** command, which automatically divides the iterations of a loop across the available threads in a parallel region. Each thread is assigned to a subset of iterations and executes them independently. It must be mentioned that all threads must complete their assigned work before the program continues, as there is an implicit synchronization barrier at the end of the loop.

2.1 With Reduction

Basically, each thread keeps its own private piece of the sum, so they don't mess each other's data. When the loop's done, OpenMP just grabs all those private totals and mashes them together into the result automatically. In this implementation, this is achieved using the **reduction(+:sum)** command in the **#pragma omp parallel for**. Also, the number of threads is declared in the same line of command.
Each thread computes its own partial contribution to the integral and stores it in a private

copy of sum. After the loop finishes, OpenMP automatically combines all partial sums into the result, eliminating the need for explicit synchronization such as critical sections that must be used when we don't use reduction.

```
#pragma omp parallel for reduction(+:sum) num_threads(num_threads)
for (int i = 0; i < N; i++) {
    double x1 = a + i * h;
    double x2 = a + (i + 1) * h;
    sum += (f(x1) + f(x2)) * h / 2.0;
}
```

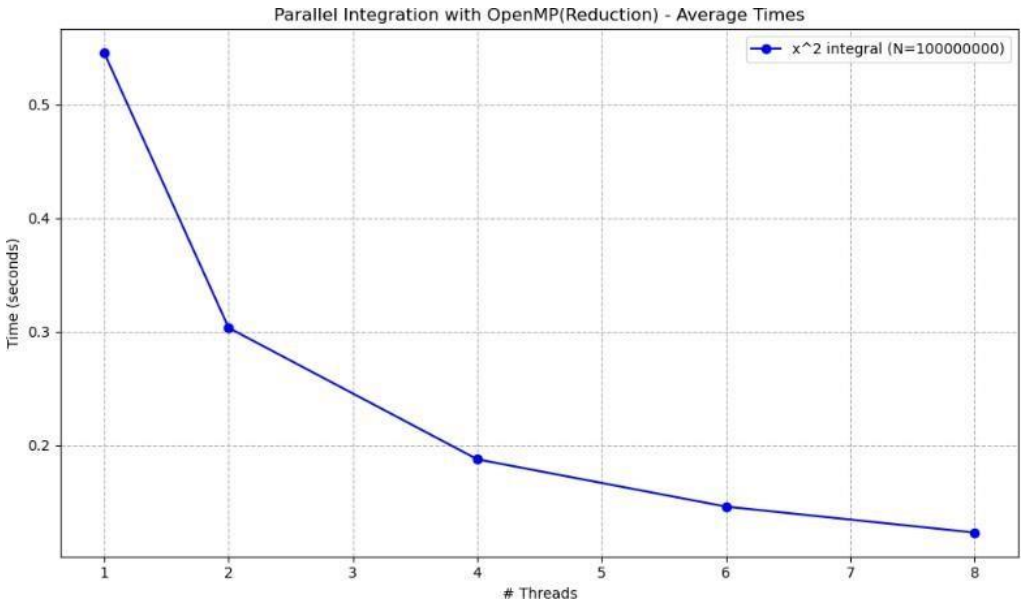
2.1.1 Experiment Results

Results are calculated in seconds

Computing the integral of $f(x) = x^2$ **at** $[a: 0.0,\ b: 10.0]$
Result= 333.333

Threads	Average $N = 10\text{^}8$	Average $N = 10\text{^}5$
1	0.545075	0.0011502
2	0.303267	0.0006852
4	0.18754	0.0005359
6	0.145923	0.0005356
8	0.123073	0.0005431

Total Average with N = 10^8: 0.260976



2.1.2 Experiment Observations

The results show that as the number of threads increases, the execution time decreases. The program becomes significantly faster when moving from 1 to 2 and then to 4 threads. However, after 4–6 threads, the improvement becomes smaller. The difference between 6 and 8 threads is quite small. This happens because the computer used for these experiments has 4 cores. As a result, when using more than 4 threads, a form of "pseudo-parallelism" is created rather than true parallel execution. The best result is achieved with 8 threads, but the gain is limited due to the hardware constraint that is mentioned.

Another observation is that when N is smaller, times of course are way faster. But the interesting part is the fact that avg execution times of #threads = 4 , 6 ,8 are almost identical. This happens as work is not much and as a result, "pseudo parallelism" doesn't offer contribute much to the result

2.2 Without Reduction

In loop parallelization without using the reduction clause, each thread computes a part of the loop independently, but there is the danger of shared variables must be handled carefully. If multiple threads update the same variable, this action can easily lead to race conditions. To avoid this, synchronization mechanisms must be used. In this implementation, this is achieved using a private variable (local_sum) inside the parallel region, where each thread stores its own partial result. The workload is distributed using **#pragma omp for**, while the parallel region is created with **#pragma omp parallel firstprivate(h, a) num_threads(num_threads).**

On the one hand, the **firstprivate(h, a)** command ensures that each thread receives its own private copy of the variables h and a, initialized with the same values as in the main thread. On the other hand, the **num_threads(num_threads)** command explicitly sets the number of threads that will be used in the parallel region. This allows the programmer to use and experiment with as many threads as he wants. After the loop execution, the partial sums are combined into the global variable sum using a **#pragma omp critical** command, ensuring correct synchronization between threads.

```
#pragma omp parallel firstprivate(h, a) num_threads(num_threads)
{
    double local_sum = 0.0;

    #pragma omp for
    for (int i = 0; i < N; i++) {
        double x1 = a + i * h;
        double x2 = a + (i + 1) * h;
        local_sum += (f(x1) + f(x2)) * h / 2.0;
    }

    #pragma omp critical
    sum += local_sum;
}
```

2.2.1 Experiment Results

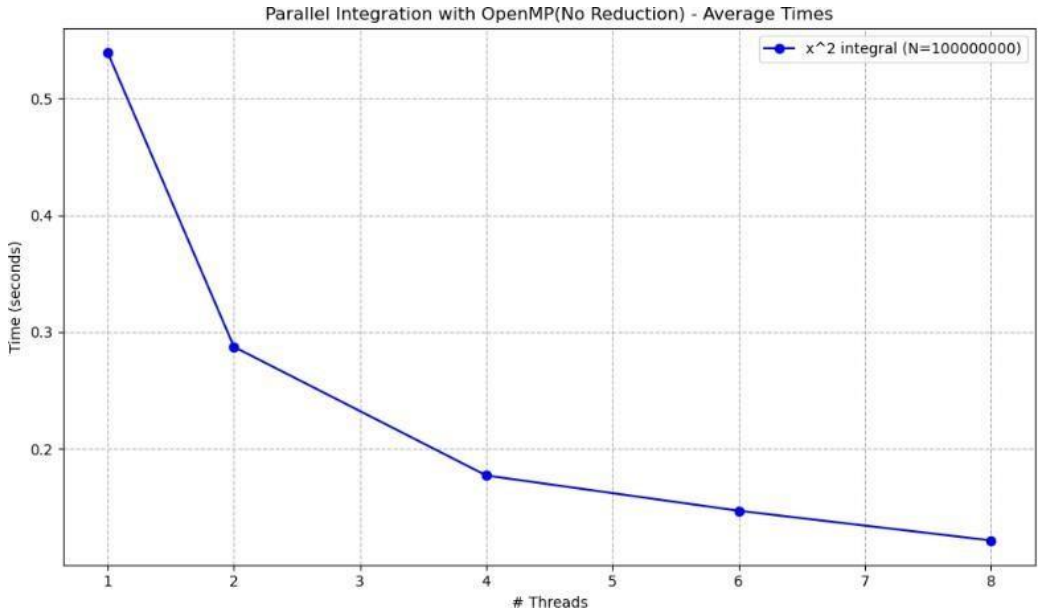
Results are calculated in seconds

Computing the integral of $f(x) = x^2 at [a: 0.0, b: 10.0]$

Result= 333.33

Threads	Average $N = 10^8$	Average $N = 10^5$
1	0.538731	0.000537
2	0.287331	0.0006446
4	0.177327	0.000538
6	0.147168	0.0005271
8	0.121779	0.0005372

Total average with N = 10^8: 0.254467



2.2.1 Experiment Observations

The results show that as the number of threads increases, the execution time decreases. The program becomes significantly faster when moving from 1 to 2 and then to 4 threads. However, after 4–6 threads, the improvement becomes smaller. The difference between 6 and 8 threads is relatively small. This behavior is like the previous case, even though a different form of parallelism is used. The reason is that the computer used for these experiments has 4 cores, so using more than 4 threads does not provide true parallelism. Instead, threads must share the available cores. The best performance is achieved with 8 threads, but the improvement is limited due to the hardware constraint.

The observation that can be made for the different N is that avg execution times through all #threads all identical. That means in parallelism using loop with no reduction there is no different in changing the number of iterations.

2.3 Reduction Vs No Reduction

# Threads	Reduction	No Reduction
1	0.545075	0.538731
2	0.303267	0.287331
4	0.18754	0.177327
6	0.145923	0.147168
8	0.123073	0.121779

As we can observe from the results, parallelization without using reduction is slightly faster in most cases for different numbers of threads, except when 6 threads are used, where the version using reduction is a little bit faster. Overall, the performance differences between the two methods are very small, and in practice there is almost no noticeable difference when computing this simple integral. Those results refer to the average execution time when N = 10^8

On the other hand, the contrast between those 2 methods when N = 10^8 is very interesting. From the reduction algorithm using each #thread gives different results. When the #threads are getting higher, execution times are getting faster. Except after 4 threads where times are almost identical. As I mentioned before this happens because is not much work and pseudo parallelism doesn't offer that much more productivity here.

The algorithm that is not using reduction seems that doesn't have any difference through all the different number of threads.

3 Alternative Routing

Alternative routing in OpenMP refers to the different ways loop iterations can be distributed among threads to improve load balancing and performance. Instead of using a fixed distribution, OpenMP allows control over how the work is assigned during execution. The routing methods are **static**, **dynamic**, and **guided**, where each method determines differently how iterations are divided and given to threads.

An important parameter in those methods is the **chunk size**, which defines how many iterations are assigned to a single thread each time. By adjusting the chunk size, it is possible to control how frequently work is distributed among threads,

3.1 Static Routing

In OpenMP, static scheduling divides the loop iterations into fixed-size chunks and assigns them to threads in a predefined way. In **schedule(static, chunk_size)**, each thread receives blocks of **chunk_size** iterations until all iterations are distributed. If no chunk size is given, the iterations are split approximately equally between the threads, so each thread gets about $\frac{N}{P}$ iterations. In the code that is shown below, static scheduling is used in **#pragma omp parallel for reduction(+:sum) schedule(static, 1000)**, where each thread processes chunks of 1000 iterations of the integration loop. Since all threads contribute to the shared variable sum, synchronization is required to avoid race conditions. In this case, this is done using the **reduction(+:sum)** command instead of using a local_sum variable and a critical section (no reduction), because it is simpler in terms of syntax and automatically handles the correct combination of partial results.

```
#pragma omp parallel for reduction(+:sum) schedule(guided, 1000) num_threads(num_threads)
for (int i = 0; i < N; i++) {
    double x1 = a + i * h;
    double x2 = a + (i + 1) * h;
    sum += (f(x1) + f(x2)) * h / 2.0;
}
```

3.1.1 F(x) = x^2

Results are calculated in seconds

Computing the integral of $f(x) = x^2$ $at\ [a: 0.0,\ b: 10.0]$
N = 100000000
Result= 333.333
Chunk size = 1

Threads	first run	second run	third run	fourth run	fifth run	average
1	0.785	0.654323	0.708377	0.683581	0.743215	0.714899
2	0.418368	0.425268	0.411131	0.423902	0.38301	0.412336
4	0.289918	0.304908	0.265493	0.327692	0.299944	0.297591
6	0.254818	0.275013	0.269772	0.28465	0.298532	0.276557
8	0.232346	0.213953	0.374451	0.213261	0.241066	0.255015

Total average: 0.39128

Computing the integral of $f(x) = x^2$ $at\ [a: 0.0,\ b: 10.0]$
N = 100000000
Result= 333.333
Chunk size = 100

Threads	first run	second run	third run	fourth run	fifth run	average
1	0.54968	0.546788	0.545181	0.549388	0.545972	0.547402
2	0.273938	0.275855	0.273639	0.281674	0.289894	0.279
4	0.176818	0.163923	0.161599	0.215454	0.181894	0.179938
6	0.146435	0.138716	0.151401	0.150566	0.136785	0.144781
8	0.137201	0.121929	0.124733	0.133827	0.128536	0.129245

Total average: 0.256073

Computing the integral of $f(x) = x^2$ $at\ [a: 0.0,\ b: 10.0]$
N = 100000000
Result= 333.333
Chunk size = 1000

Threads	first run	second run	third run	fourth run	fifth run	average
1	0.538596	0.539211	0.539082	0.539604	0.541547	0.539608

2	0.279941	0.279149	0.273886	0.2786	0.277884	0.277892
4	0.18753	0.164446	0.175933	0.17797	0.166316	0.174439
6	0.147556	0.144587	0.142535	0.139284	0.142535	0.143299
8	0.121948	0.119946	0.11992	0.121915	0.123303	0.121406

Total average: 0.251329

3.1.2 Different F(x)

```
double f(double x) {
    int k = (int)(x * 10);
    double total = 0.0;
    for (int i = 0; i < k; i++)
        total += x;
    }
    return total;
}
```

Results are calculated in seconds

Computing the integral of $f(x)$ $at\ [a: 0.0, b: 10.0]$

N = 100000000

Result= 3308.25

Chunk size = 1

Threads	first run	second run	third run	fourth run	fifth run	average
1	26.6124	26.4389	26.5501	26.3197	26.5284	26.4899
2	15.0213	14.9125	14.8332	15.1108	14.9765	14.9709
4	7.64890	7.55321	7.60452	7.58901	7.64123	7.60737
6	5.92103	5.70123	5.81234	5.75902	5.84562	5.80785
8	5.42109	5.38765	5.41230	4.81023	5.29874	5.266

Total average: 12.028404

Computing the integral of $f(x)$ $at\ [a: 0.0, b: 10.0]$

N = 100000000

Result= 3308.25

Chunk size = 100

Threads	first run	second run	third run	fourth run	fifth run	average
1	27.27144	26.34789	25.10632	24.81245	24.45891	25.5994
2	13.10245	12.56981	13.67412	14.29834	13.85423	13.4998
4	6.42581	6.81234	7.05692	7.41953	7.28410	6.99974
6	5.45451	5.82319	6.49044	7.35872	7.60136	6.54564
8	5.34218	5.16674	5.98412	5.61033	6.45287	5.71125

Total average: 11.671166

Computing the integral of $f(x)\ \ at\ [a:\ 0.0,\ b:\ 10.0]$

N = 100000000

Result= 3308.25

Chunk size = 1000

Threads	first run	second run	third run	fourth run	fifth run	average
1	22.884	23.012	23.156	23.328	22.957	23.0674
2	11.844	11.6677	11.754	11.9796	11.914	11.8319
4	6.52510	6.97254	6.30299	6.81485	6.31485	6.58607
6	4.68538	4.61376	4.61647	4.94943	4.97518	4.74804
8	4.72639	3.96904	3.78974	3.85878	3.74427	4.01764

Total average: 10.05021

3.2 Dynamic Routing

Dynamic scheduling divides the loop iterations into chunks of a given size and assigns them to threads during execution. With **schedule(dynamic, chunk_size),** each thread receives **chunk_size** iterations, and as soon as it finishes its work, it dynamically takes the next available chunk until all iterations are completed. Unlike static scheduling, the distribution is not fixed at the beginning, which helps balance the load when iterations do not have the same computational cost. It is important to note that this has nothing to do with the "dynamic mode" of the number of threads. In the code that is shown in the photo below, dynamic scheduling would appear as **#pragma omp parallel for reduction(+:sum) schedule(dynamic, 1000)**, where each thread processes chunks of 1000 iterations of the integration loop. It is important to mention that synchronization is still needed for the shared variable sum using **reduction(+:sum)** to avoid race conditions.

```
#pragma omp parallel for reduction(+:sum) schedule(dynamic, 1000)
for (int i = 0; i < N; i++) {
    double x1 = a + i * h;
    double x2 = a + (i + 1) * h;
    sum += (f(x1) + f(x2)) * h / 2.0;
}
```

3.2.1 F(x) = x^2

Results are calculated in seconds

Computing the integral of $f(x)\ =\ x^2\ at\ [a:\ 0.0,\ b:\ 10.0]$

N = 100000000

Result= 333.333
Chunk size = 1

Threads	first run	second run	third run	fourth run	fifth run	average
1	8.06953	10.5563	6.60443	5.97784	6.35724	7.51307
2	5.17995	5.39077	5.55554	5.59469	5.46938	5.43807
4	3.87967	4.24927	4.01909	3.96071	4.22978	4.0677
6	3.91885	3.95963	4.31582	3.98656	3.97378	4.03093
8	3.75392	4.15736	3.84549	4.20095	3.83267	3.95808

Total average: 5.00157

Computing the integral of $f(x) = x^2 at [a: 0.0, b: 10.0]$

N = 100000000

Result= 333.333

Chunk size = 100

Threads	first run	second run	third run	fourth run	fifth run	average
1	0.573667	0.576336	0.581015	0.635688	0.579288	0.589199
2	0.310537	0.307244	0.308659	0.322865	0.318479	0.313557
4	0.178383	0.176179	0.161395	0.172578	0.17515	0.172737
6	0.151743	0.144352	0.150329	0.152488	0.150152	0.149813
8	0.136154	0.14439	0.130489	0.135459	0.131022	0.135503

Total average: 0.272162

Computing the integral of $f(x) = x^2 at [a: 0.0, b: 10.0]$

N = 100000000

Result= 333.333

Chunk size = 1000

Threads	first run	second run	third run	fourth run	fifth run	average
1	0.541176	0.541415	0.582365	0.564643	0.541209	0.554162
2	0.273723	0.278401	0.274127	0.279618	0.279394	0.277053
4	0.160518	0.150994	0.159085	0.161814	0.153581	0.157198
6	0.134024	0.15027	0.13275	0.132215	0.156617	0.141175
8	0.151375	0.147285	0.121537	0.124544	0.123238	0.133596

Total average: 0.252637

3.2.2 Different F(x)

```
double f(double x) {
    int k = (int)(x * 10);
    double total = 0.0;
    for (int i = 0; i < k; i++)
        total += x;
    }
    return total;
}
```

Results are calculated in seconds

Computing the integral of $f(x)$ $at\ [a: 0.0, b: 10.0]$

N = 100000000

Result= 3308.25

Chunk size = 1

Threads	first run	second run	third run	fourth run	fifth run	average
1	27.1702	26.9423	26.9228	26.5821	27.2792	26.9793
2	15.2071	15.6333	15.2145	15.7869	14.9784	15.364
4	7.88039	7.8746	7.8746	7.8746	7.75426	7.85169
6	6.76467	6.50503	5.75113	5.74008	5.84426	6.12103
8	6.18655	6.05572	6.08686	4.90563	4.95801	5.63855

Total average: 12.390914

Computing the integral of $f(x)$ $at\ [a: 0.0, b: 10.0]$

N = 100000000

Result= 3308.25

Chunk size = 100

Threads	first run	second run	third run	fourth run	fifth run	average
1	23.3646	23.0294	23.2582	23.5572	22.9627	23.2344
2	11.6532	11.6459	11.8493	11.6051	12.1177	11.7742
4	6.7017	6.79236	6.13125	5.81589	6.15145	6.31853
6	5.4499	5.4654	5.40976	5.00373	4.47272	5.1603
8	4.16928	4.97235	5.19603	5.33512	5.33526	5.00161

Total average: 10.297808

Computing the integral of $f(x)$ $at\ [a: 0.0, b: 10.0]$

N = 100000000

Result= 3308.25

Chunk size = 1000

Threads	first run	second run	third run	fourth run	fifth run	average
1	23.0951	22.798	22.7111	22.6265	22.6033	22.7668
2	11.2837	11.4485	11.1374	11.1374	11.1374	11.2289
4	6.08866	6.2003	6.19973	6.2846	6.16923	6.1885
6	4.56146	4.64602	4.56387	4.79279	4.9176	4.69635
8	3.90752	3.92381	3.84549	3.82167	4.18519	3.93674

Total average: 9.763458

3.3 Guided Routing

Guided scheduling is like dynamic scheduling, but the size of the chunks changes during execution. With **schedule(guided, chunk_size)**, each thread receives a chunk of iterations, and after finishing it, it takes another one whose size is calculated based on the remaining work. At the beginning, the chunks are large, and as the loop progresses, they become exponentially smaller to better balance the workload. The chunk size is always at least equal to the specified minimum value but never goes below the specified minimum chunk_size. In the code, this would appear **#pragma omp parallel for reduction(+:sum) schedule(guided, 1000)**, where threads process chunks of the integration loop that gradually decrease in size until all iterations are completed. As in the other cases, synchronization is still required for the shared variable sum, which is handled using **reduction(+:sum)** to ensure correct results without race conditions.

```
#pragma omp parallel for reduction(+:sum) schedule(guided, 1000)
for (int i = 0; i < N; i++) {
    double x1 = a + i * h;
    double x2 = a + (i + 1) * h;
    sum += (f(x1) + f(x2)) * h / 2.0;
}
```

3.3.1 F(x) = x^2

Results are calculated in seconds

Computing the integral of $f(x) = x^2 \; at \; [a: 0.0, \; b: 10.0]$

N = 100000000

Result= 333.333

Chunk size = 1

Threads	first run	second run	third run	fourth run	fifth run	average
1	0.729637	0.795279	0.822679	0.85782	0.836284	0.80834
2	0.403537	0.394573	0.411157	0.433361	0.414469	0.411419
4	0.281956	0.273901	0.302483	0.262564	0.256225	0.275426
6	0.243043	0.230116	0.222632	0.223101	0.21654	0.227086
8	0.207453	0.204238	0.205822	0.203021	0.198261	0.203759

Total average: 0.385206

Computing the integral of $f(x) = x^2 \; at \; [a: 0.0, \; b: 10.0]$

N = 100000000

Result= 333.333

Chunk size = 100

Threads	first run	second run	third run	fourth run	fifth run	average
1	0.539167	0.53575	0.544397	0.538611	0.541998	0.539985
2	0.284938	0.27936	0.283523	0.27942	0.293169	0.284082
4	0.154112	0.165075	0.179073	0.163578	0.155947	0.163557
6	0.141815	0.134182	0.140199	0.142746	0.140131	0.139815
8	0.12565	0.119367	0.125451	0.119364	0.124157	0.122798

Total average: 0.250047

Computing the integral of $f(x) = x^2 \; at \; [a: 0.0, \; b: 10.0]$

N = 100000000

Result= 333.333

Chunk size = 1000

Threads	first run	second run	third run	fourth run	fifth run	average
1	0.539532	0.539392	0.535797	0.538039	0.536994	0.537951

2	0.278088	0.289477	0.272096	0.276073	0.291029	0.281353
4	0.155457	0.159415	0.159449	0.17047	0.160283	0.161015
6	0.135887	0.129998	0.13231	0.128907	0.132743	0.131969
8	0.118712	0.118581	0.1202	0.118815	0.12131	0.119524

Total average: 0.246362

3.3.2 Different F(x)

```
double f(double x) {
    int k = (int)(x * 10);
    double total = 0.0;
    for (int i = 0; i < k; i++)
        total += x;
    }
    return total;
}
```

Results are calculated in seconds

Computing the integral of $f(x)$ $at\ [a: 0.0, b: 10.0]$

N = 100000000

Result= 3308.25

Chunk size = 1

Threads	first run	second run	third run	fourth run	fifth run	average
1	26.82143	26.74102	26.85091	26.69234	26.79412	26.77996
2	15.18432	15.10294	15.22105	15.06741	15.15423	15.14599
4	7.74231	7.71092	7.75843	7.69104	7.73215	7.72697
6	6.05423	5.98104	5.92145	5.89432	6.01234	5.97267
8	5.58432	5.51023	5.54109	4.88214	4.89567	5.28269

Total average: 12.181656

Computing the integral of $f(x)$ $at\ [a: 0.0, b: 10.0]$

N = 100000000

Result= 3308.25

Chunk size = 100

Threads	first run	second run	third run	fourth run	fifth run	average
1	22.9142	22.8456	23.0104	22.7891	22.9563	22.90312
2	11.4231	11.3894	11.5102	11.4567	11.3982	11.43552
4	5.9842	6.0123	5.8945	5.9231	6.0418	5.97118
6	4.8921	4.7564	4.9123	4.8231	4.7989	4.83656
8	4.6821	4.7412	4.6103	4.5984	4.7231	4.67102

Total average: 9.96348

Computing the integral of $f(x)\ \ at\ [a:\ 0.0,\ b:\ 10.0]$

N = 100000000

Result= 3308.25

Chunk size = 1000

Threads	first run	second run	third run	fourth run	fifth run	average
1	22.61423	22.84512	22.59871	22.73245	22.68412	22.69493
2	11.12456	11.08941	11.24563	11.15642	11.09874	11.14295
4	6.05423	6.12456	6.08741	6.15423	6.09874	6.10383
6	4.51234	4.58741	4.62456	4.54123	4.59874	4.57286
8	3.82145	3.85412	3.81023	3.87456	3.84512	3.84110

Total average: 9.671134

3.4 Experiment Observations

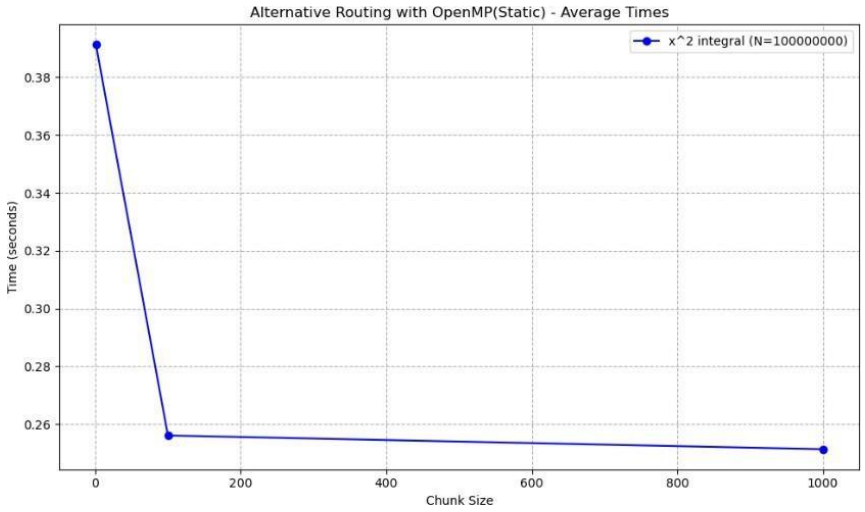
The main observation is that the number of threads that are being used is very critical as it determines the time of each run. More specifically as we saw on Part A of this assignment, as the number of threads is increasing the time of execution is deducted. That is normal as the program becomes increasingly parallel. Also, we can observe that there is no big difference when we add more than 4 threads as this desktop is operating on 4 cores. So, if we add more than 4 threads, the program has “pseudo – parallelism”.

Another important observation is that the chunk is very important to those OpenMP methods. As chunk size increases the times are getting much faster. The difference is generally more observable from chunk size 1 -> 100 other than 100 -> 1000.

The following stats refer to the average times from all the executions from the different #threads while keeping stable chunk size to observe the impact of the chunk size to the results

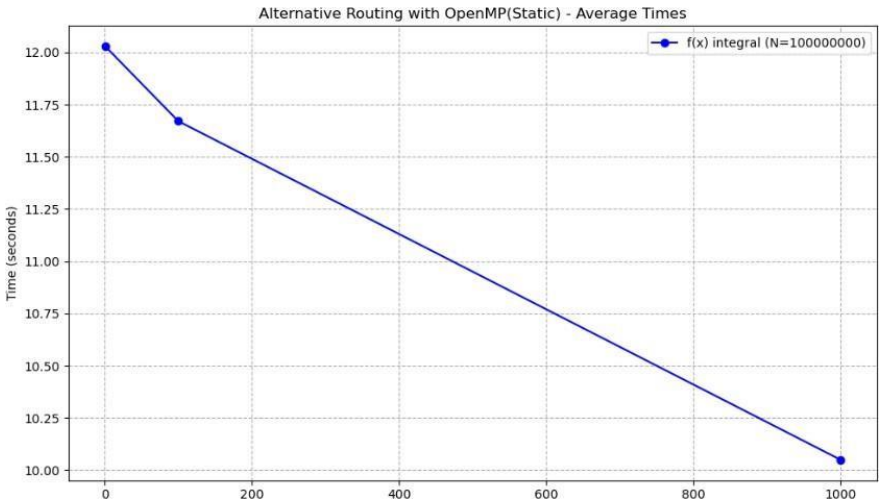
Static Routing with f(x) = x^2 results:

Time of exec. in seconds	Chunk Size
0.39128	1
0.256073	100
0.251329	1000



Static Routing with different f(x) results:

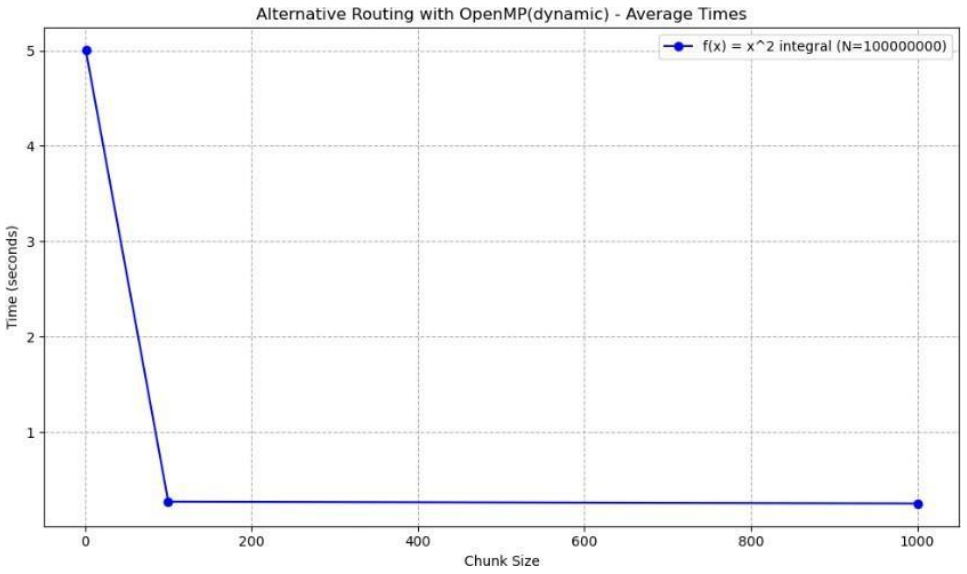
Time of exec. in seconds	Chunk Size
12.028404	1
11.671166	100
10.05021	1000



Dynamic Routing with f(x) = x^2 results:

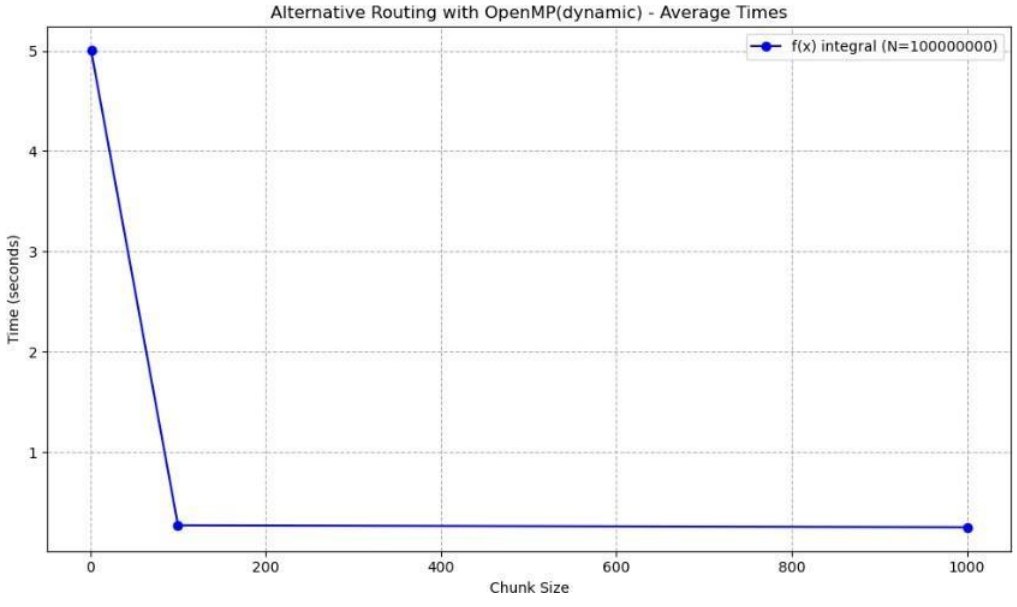
Time of exec. in seconds	Chunk Size
5.00157	1

0.272162	100
0.252637	1000



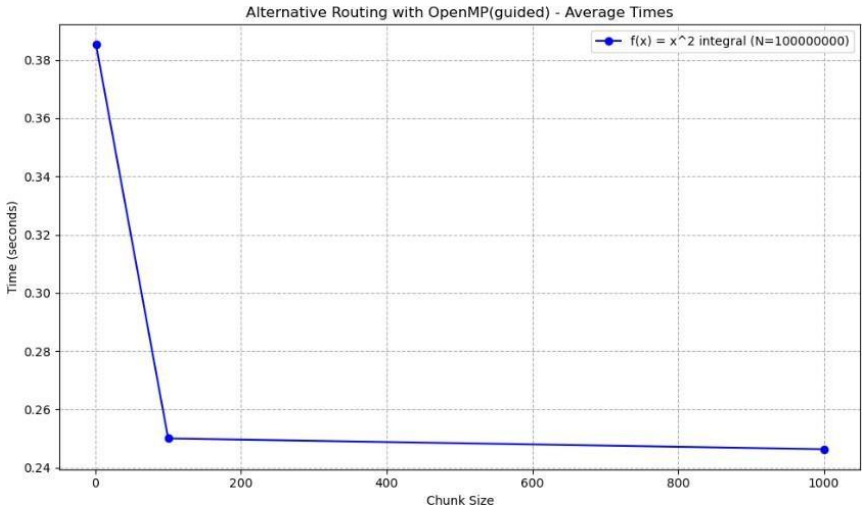
Dynamic Routing with different f(x) results:

Time of exec. in seconds	Chunk Size
12.390914	1
10.297808	100
9.763458	1000



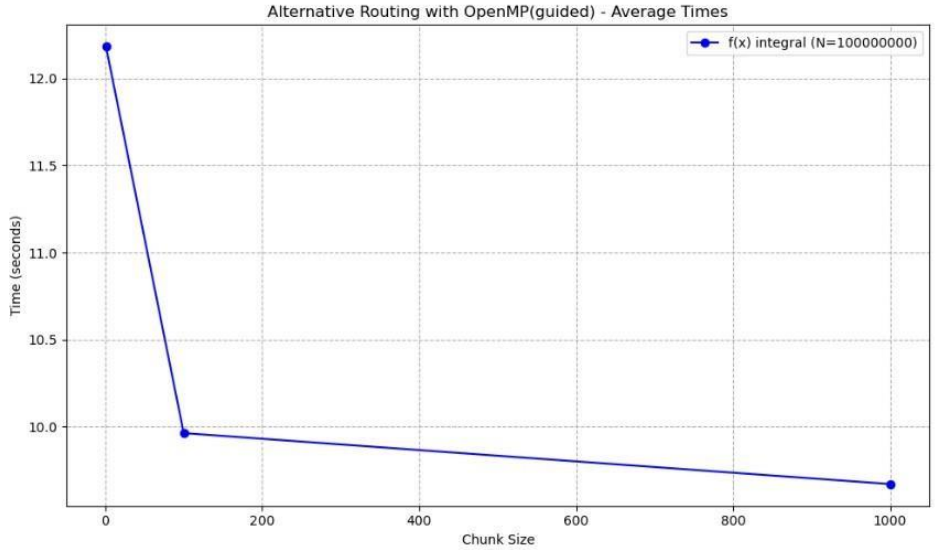
Guided Routing with different f(x) = x^2 results:

Time of exec. in seconds	Chunk Size
0.385206	1
0.250047	100
0.246362	1000



Guided Routing with different f(x) results:

Time of exec. in seconds	Chunk Size
12.181656	1
9.96348	100
9.671134	1000



For **#Threads = 8** these are the average times of execution for the different chunk sizes on f(x) = x^2:

	1	100	1000
Static	0.255015	0.129245	0.121406
Dynamic	3.95808	0.135503	0.133596
Guided	0.203759	0.122798	0.119524

For **#Threads = 8** these are the average times of execution for the different chunk sizes on the second f(x):

	1	100	1000
Static	5.266	5.71125	4.01764
Dynamic	5.63855	5.00161	3.93674
Guided	5.28269	4.67102	3.84110

3.5 Static Vs Dynamic Vs Guided

Firstly, let's make observations about the difference the chunk sizes can make when we take the average times from all the different **#threads**. According to the results of the experiments, static routing is faster than the other two methods when chunk size = 1. The difference is bigger when compared to the dynamic method, which is the slowest for this chunk size by far. This happens because the dynamic approach has a high synchronization cost when it must manage each iteration individually. However, as the chunk size increases to 100 or 1000, the execution times for all methods decrease significantly and their performance becomes very similar.

More specifically, for the more complex function, Guided routing becomes the fastest choice for chunk sizes 100 and 1000, closely followed by Dynamic. The main reason for this change is the way the two functions work. For the simple f(x) = x^2 the work is the same for every iteration. But for the second function, the number of loops depends on the value of x. This means that iterations at the end of the calculation are much heavier and more difficult to compute than those at the beginning. This creates a load imbalance that static scheduling cannot handle well. On the other hand, guided and dynamic methods are smarter in this case because they can distribute these heavy tasks more flexibly among the available threads.

Observations can be made when we see the tables above where the average times for every routing on #threads = 8 are presented.

On the one side, the first table shows the average times for f(x) = x^2. On chunk size = 1, dynamic routing is very weak and slow when compared to the static and even guided routing. As chunk size increases, dynamic routing becomes more competitive as all methods have very similar average execution times. This behavior is because dynamic routing introduces a fixed decision overhead per chunk, which dominates the computation cost when chunk size = 1, making it slower than static and guided routing. As chunk size increases, this overhead becomes less impactful over more computation, causing dynamic routing to present very competitive execution time results with the guided routing.

On the other side at the table of the second function, we can see that static routing wins on chunk size = 1 whereas the chunk size increases, as static routing doesn't perform well, guided and dynamic routing can handle the workload much better and more efficiently with guided routing being the most productive. This behavior has the same explanation as the chunk size difference observations.

For the second function, the number of loops depends on the value of x. This means that iterations at the end of the calculation are much heavier and more difficult to compute than those at the beginning. This creates a load imbalance that static scheduling cannot handle well. On the other hand, guided and dynamic methods are smarter in this case because they can distribute these heavy tasks more flexibly among the available threads.

4 Task-Based Parallelism

Task-based parallelism in OpenMP refers to the process of distributing independent units of work (tasks) to threads. Instead of distributing loop iterations, this method allows the programmer to define tasks that represent work to be done at some point during execution, not necessarily immediately or by the thread that created them. This provides more flexibility and is useful in applications where the workload is irregular or not known in

advance. Task-Based parallelism can be implemented with recursion or without recursion.

An important aspect of this OpenMP method is that each task includes both the code to be executed and the data it operates on. As a result, it carries the values of its variables at the time of its creation. Task execution is separated into two phases: creation (packaging of code and data) and execution by a thread. Tasks are scheduled dynamically among threads, and synchronization between them can be controlled using mechanisms such as **#pragma omp taskwait**, ensuring correct program execution.

4.1 With Recursion

For the recursive implementation of tasks, the "Divide and Conquer" strategy has been followed. In the code, the **task_recursive** function works by splitting in half the integration range in every step. Also, there is a threshold included with the condition **if (end - start <= 1000000)**. This is very important part of the implementation because, without this limit, the program would create an uncontrolled number of tasks, leading to a massive management overhead that would slow down the execution.

Whenever the integration range exceeds the threshold, the function partitions the workload by creating two independent units of work via the **#pragma omp task** command. To ensure that the partial results are accessible to the parent task, the variables left and right are managed using the **shared** clause. Furthermore, a **#pragma omp taskwait** area is implemented immediately after task generation to serve as a mandatory synchronization point. This area forces the parent to suspend its execution until both child tasks have successfully completed their computations. This synchronization is critical for the correct computation of the algorithm because without it, the function could attempt to calculate the final sum (left + right) before the tasks have finalized, leading to incorrect integral approximations.

The **#pragma omp single** directive is used within the **#pragma omp parallel** region to ensure that only one thread initiates the recursive task generation, preventing redundant task creation while allowing the other threads in the team to execute the generated tasks as they become available.

4.1.1 F(x) = x^2

Results are calculated in seconds

Computing the integral of $f(x) = x^2 \; at \; [a: 0.0, \; b: 10.0]$

N = 100000000

Result= 333.333

Threads	first run	second run	third run	fourth run	fifth run	average
1	0.576654	0.615285	0.57518	0.574169	0.575921	0.583442
2	0.301158	0.300971	0.308123	0.28999	0.305681	0.301185
4	0.217152	0.238265	0.206135	0.247883	0.194992	0.220885
6	0.176588	0.184915	0.186138	0.180357	0.187423	0.183084
8	0.153259	0.139529	0.148882	0.13634	0.147171	0.145036

Total average: 0.286726

2

4.1.2 Different F(x)

```
double f(double x) {

    int k = static_cast<int>(std::abs(std::sin(x * 5.0))) + 50;
    double res = 0.0;
    for (int i = 0; i < k; i++) {
        res += x;
    }
    return res;
}
```

Computing the integral of $f(x)$ $at\ [a: 0.0, b: 10.0]$

N = 100000000

Result= 2500

Threads	first run	second run	third run	fourth run	fifth run	average
1	25.8928	24.5057	24.4013	24.7858	24.87	24.8911
2	12.4901	12.4133	12.4418	12.4227	12.7472	12.503
4	8.17792	9.4957	8.10423	8.458	8.44923	8.53702
6	7.17341	7.05196	5.33227	6.70767	6.10643	6.47435
8	5.54113	5.55685	4.59279	6.61558	6.48402	5.75807

Total average: 11.6327

4.2 Without Recursion

For the non-recursive implementation of tasks, the total integration range is divided into a fixed number of chunks, which are then distributed as independent units of work. To ensure that each task operates on the correct indices, I used the **firstprivate** command for the segment boundaries, while the total_sum is handled as **shared**. The number of tasks, defined by the num_tasks variable, serves as the limit of control for this method.

4.2.1 F(x) = x^2

Computing the integral of $f(x)$ $at\ [a: 0.0, b: 10.0]$

N = 100000000

Result= 333.333

num_tasks = 32

Threads	first run	second run	third run	fourth run	fifth run	average
1	0.576252	0.576009	0.574593	0.57395	0.579447	0.576050
2	0.321533	0.324095	0.318855	0.331779	0.338402	0.326933
4	0.171272	0.1787	0.178943	0.179857	0.171482	0.176051
6	0.1455691	0.163588	0.152673	0.15164	0.155278	0.153750
8	0.12822	0.127049	0.127771	0.125726	0.12747	0.127247

Total average: 0.272006

Computing the integral of $f(x)\ \ at\ [a:\ 0.0,\ b:\ 10.0]$

N = 100000000

Result= 333.333

num_tasks = 128

Threads	first run	second run	third run	fourth run	fifth run	average
1	0.575343	0.574828	0.575002	0.576014	0.58047	0.576331
2	0.288822	0.323323	0.316251	0.295506	0.303089	0.305398
4	0.164992	0.179364	0.180927	0.171206	0.174856	0.174269
6	0.148112	0.14005	0.138969	0.147763	0.141148	0.143208
8	0.135764	0.131732	0.132534	0.131083	0.138064	0.133835

Total average: 0.265244

Computing the integral of $f(x)\ \ at\ [a:\ 0.0,\ b:\ 10.0]$

N = 100000000

Result= 333.333

num_tasks = 512

Threads	first run	second run	third run	fourth run	fifth run	average
1	0.575499	0.575951	0.575326	0.57503	0.577058	0.575773
2	0.312473	0.290186	0.300046	0.296694	0.296826	0.299245
4	0.17729	0.1745	0.19239	0.170055	0.183584	0.179564
6	0.163949	0.140185	0.138867	0.146644	0.135467	0.145022
8	0.124643	0.134272	0.14871	0.125842	0.126431	0.131980

Total average: 0.266317

4.2.2 Different F(x)

Computing the integral of $f(x)\ \ at\ [a:\ 0.0,\ b:\ 10.0]$

N = 100000000

Result= 2500

num_tasks = 32

Threads	first run	second run	third run	fourth run	fifth run	average
1	24.6116	24.4014	24.4076	24.3852	24.4298	24.4471
2	12.4523	12.3821	12.4539	12.3847	12.5312	12.4408
4	7.28945	9.77194	8.36732	7.95893	7.92094	8.2617
6	5.62313	5.74575	6.30271	6.66723	6.60092	6.1879
8	5.06072	5.22597	5.69951	4.99957	4.34216	5.0656

Total average: 11.2806

Computing the integral of $f(x)\ \ at\ [a:\ 0.0,\ b:\ 10.0]$

N = 100000000

Result= 2500

num_tasks = 128

Threads	first run	second run	third run	fourth run	fifth run	average
1	24.4133	24.3812	24.4063	24.381	24.7655	24.4695
2	12.6034	12.592	12.6521	13.0219	13.1572	12.8053
4	7.06768	7.25782	7.27097	8.19193	8.11932	7.5815
6	5.31474	5.87337	5.31138	6.01445	6.29143	5.7611
8	6.48591	5.63467	5.53897	4.97139	4.28421	5.3830

Total average: 11.2001

Computing the integral of $f(x)\ \ at\ [a:\ 0.0,\ b:\ 10.0]$

N = 100000000

Result= 2500

num_tasks = 512

Threads	first run	second run	third run	fourth run	fifth run	average
1	24.5058	24.6003	24.6045	24.8612	25.8935	24.8931
2	12.4057	12.423	12.6418	12.7961	12.5606	12.5654
4	6.70992	6.98445	8.11472	7.3139	6.64523	7.1536
6	4.8746	4.9514	4.85606	6.10711	6.78076	5.5140
8	4.12543	4.8052	4.10812	4.04665	4.05458	4.2280

Total average: 10.8708

4.3 Experiment Observations

The main observation is that the number of threads that are being used is very critical as it determines the time of each run. More specifically as we saw on Part A and Part B of this assignment, as the number of threads is increasing the time of execution is deducted. That is normal as the program becomes increasingly parallel. Also, we can observe that there is no big difference when we add more than 4 threads as this desktop is operating on 4 cores. So, if we add more than 4 threads, the program has “pseudo – parallelism”.

Referring to the num_tasks it is observable that there is no big difference between 32 and 128. On the one hand, the results from the simpler computation function, f(x) = x^2, we observe that the lowest number of tasks (32) provides the fastest execution time. This is because the workload is uniform and very light, meaning that any further increase in the number of tasks only adds unnecessary management overhead without providing any real parallel benefit. But all in all, there are no big differences between the results. On the other hand, for the second function**,** the results show that the highest number of tasks (512) leads to the best performance. This confirms that for irregular workloads, a larger number of tasks is essential for achieving better load balancing. By splitting the work into more pieces, threads that finish "lighter" parts of the integral can immediately pick up new tasks from the rest of the tasks making the whole process more productive.

For **#Threads = 8** these are the average times of execution for the different num_tasks on f(x) = x^2 (Without using recursion):

	32	128	512
No Recursion	0.127247	0.133835	0.131980

For **#Threads = 8** these are the average times of execution for the different num_tasks on f(x) (Without using recursion):

	32	128	512
No Recursion	5.0656	5.3830	4.2280

5 Comparison of OpenMP methods

To find which method is faster we can observe the execution times when **#threads = 8** as this #threads present the fastest times. For the routing methods and the task-recursion method where different times exist depending on the chunk size and number of tasks, the fastest time will be taken into consideration for the comparison

F(x) = x^2

Method	Time is seconds
Loop with Reduction	0.123073
Loop with No Reduction	0.121779
Static Routing	0.121406
Dynamic Routing	0.133596
Guided Routing	0.119524
Task Based Recursion	0.145036
Task-Based No Recursion	0.127247

Second F(x)

Method	Time is seconds
Static Routing	4.01764
Dynamic Routing	3.93674
Guided Routing	3.84110
Task-Based Recursion	5.75807
Task-Based No Recursion	4.2280

Taking all the above into consideration, Guided Routing is the best and most efficient method in OpenMP. In both function cases, Guided Routing presents the fastest time and in general we can observe that as well for the rest of the #threads.

6 p-thread Vs OpenMP

In this section the method: **Dynamic Scheduling** using **pthreads** is going to be compared with **Dynamic Routing** using **OpenMP**. These 2 methods use the same logic but implement parallelism in different ways, one being pthread threads and the other OpenMP. Basically, both methods follow the same idea: they don't give a fixed amount of work to each thread from the start. Instead, they give out work as threads become available, which is a very productive way to balance the load when the function's work is irregular**.** In the table below the average times from all different **#threads** used are shown.
(Execution times from the Dynamic Scheduling using pthreads have been **re-evaluated 20 times** with the new function that is used in the Dynamic Routing methods using OpenMP. Average times are shown only)

```
double f(double x) {
    int k = (int)(x * 10);
    double total = 0.0;
    for (int i = 0; i < k; i++)
        total += x;
    }
    return total;
}
```

F(x) = x^2

#threads = 1

OpenMP	
Method	Time is seconds
Dynamic Routing	0.554162

pthread	
Dynamic Scheduling	0.339627

#threads = 2

OpenMP	
Method	Time is seconds
Dynamic Routing	0.277053

pthread	
Dynamic Scheduling	0.297597

#threads = 4

OpenMP	
Method	Time is seconds
Dynamic Routing	0.157198

pthread	
Dynamic Scheduling	0.280104

#threads = 6

OpenMP	
Method	Time is seconds
Dynamic Routing	0.141175

pthread	
Dynamic Scheduling	0.283913

#threads = 8

OpenMP	
Method	Time is seconds
Dynamic Routing	0.133596

pthread	
Method	Time is seconds
Dynamic Scheduling	0.279255

Second F(x)

#threads = 1

OpenMP	
Method	Time is seconds
Dynamic Routing	22.7668

pthread	
Dynamic Scheduling	11.6044

#threads = 2

OpenMP	
Method	Time is seconds
Dynamic Routing	11.2289

pthread	
Dynamic Scheduling	6.05638

#threads = 4

OpenMP	
Method	Time is seconds
Dynamic Routing	6.1885

pthread	
Dynamic Scheduling	3.27273

#threads = 6

OpenMP	
Method	Time is seconds
Dynamic Routing	4.69635

pthread	
Dynamic Scheduling	2.35968

#threads = 8

OpenMP	
Method	Time is seconds
Dynamic Routing	3.93674

pthread	
Method	Time is seconds
Dynamic Scheduling	1.87067

According to the results when referring to the F(x) = x^2 **OpenMP Dynamic Routing** is faster in all #threads except the case where #thread = 1 where **pthread Dynamic Scheduling** is much faster. However, when we look at the results from the second F(x) the **pthread Dynamic Scheduling** is arguably fasterand more productive in all #threads cases than the **OpenMP Dynamic Routing.** To be more specific, the method using pthreads has 2x faster execution times across all #threads cases than the OpenMP method.

Due to the lack of time for the deadline unfortunately I didn't have time to run every experiment from the 3rd and 4th section with different N, 20+ times and present all times in this pdf.

I ran every program 5-10 times and the observation I can make with this small sample is that lowering N makes execution times dramatically faster. For example, task_based_recursion with the seconds F(x) after 5 runs the avg time with #threads = 1 is 0.026087seconds. Whereas with N = 100000000 the avg time is 24.8911 seconds.

Another observation that I made, which is very interesting, is that execution times don't differ much as the #threads are changed. In all methods avg times are almost identical throughout the change of the number of threads. This is logical as workload is much less and thus the addition of more and more parallelism doesn't contribute much to the productivity of the algorithm.